
PEDIDOS DE VENTA

Guia de desarrollo para continuar el proyecto Flutter

Arquitectura, Velneo, modelos, pantallas, filtros y comandos practicos.

Proyecto pedidos_venta

Guia del proyecto Pedidos de Venta

1. Objetivo

Esta aplicacion Flutter consulta pedidos de venta en Velneo, autentica usuarios mediante USR_M, valida el contacto comercial asociado en ENT_M y muestra los pedidos de VTA_PED_G que pertenecen al comercial del pedido (VTA_PED_G.CMR).

La aplicacion trabaja principalmente con datos en memoria. No utiliza SQLite ni una base local persistente.

2. Como arrancar el proyecto

Desde la carpeta raiz:

```
flutter pub get
flutter analyze
flutter test
flutter run -d windows
```

Para ver los dispositivos disponibles:

```
flutter devices
```

La aplicacion tambien puede ejecutarse en Android, Chrome o Windows si el dispositivo esta preparado:

```
flutter run -d chrome
flutter run -d windows
flutter run -d <id-del-dispositivo>
```

3. Donde se programa cada cosa

lib/main.dart

Punto de entrada. Aqui se registran:

- AuthState, estado de sesion.
- OrderRepository, cache de pedidos en RAM.
- Rutas de login, lista, detalle y formulario.

No pongas aqui llamadas a Velneo ni reglas de negocio.

lib/core/config.dart

Configuracion de Velneo:

- URL base.

- API key.
- Endpoints.
- Nombres de campos de USR_M y ENT_M.
- Tamano de pagina.

Los endpoints actuales son:

```
USR_M      -> usuarios
ENT_M      -> clientes y comerciales
VTA_PED_G  -> cabeceras de pedidos de venta
VTA_PED_LIN_G -> lineas de pedidos de venta
ART_M      -> articulos
ALM_M      -> almacenes
SER_M      -> series
FPG_M      -> formas de pago
```

lib/core/api_client.dart

Cliente HTTP unico de la aplicacion.

Responsabilidades:

- Construir la URL.
- Añadir api_key.
- Ejecutar GET, POST, PUT y DELETE.
- Decodificar JSON.
- Convertir errores Velneo en ApiException.
- Extraer listas y registros de las respuestas envueltas de Velneo.

No cambies las pantallas para construir peticiones HTTP. Las peticiones deben pasar por ApiClient y PedidosService.

lib/api_service.dart

Capa de servicios de Velneo. Aqui se programa que endpoint se consulta y que payload se envia.

Metodos principales:

- authenticateUser(): valida usuario y contraseña.
- _getContact(): obtiene el contacto asociado.
- list(): obtiene una pagina de pedidos.
- listAll(): descarga paginas de pedidos.
- getByld(): obtiene cabecera y lineas.
- create(): crea una cabecera.
- update(): actualiza una cabecera.
- enviarLineas(): crea o actualiza lineas.
- remove(): elimina una cabecera.
- getArticulos(), getSeries(), getAlmacenes(), getFormasPago(): carga maestros.

Si Velneo cambia un nombre de campo o endpoint, este es uno de los primeros archivos que debes revisar.

lib/models.dart

Modelos Freezed y conversion de JSON.

Modelos principales:

- Pedido: cabecera y datos relacionados.
- LineaPedido: articulo, cantidades, precio, descuento e IVA.
- Cliente: datos del cliente.
- User: usuario, rol y contacto asociado.
- OpcionMaestra: opciones de desplegables.

Las funciones `_normalizePedidoJson()` y `_normalizeLineaJson()` traducen nombres Velneo a nombres Dart.

Despues de cambiar `models.dart`, ejecuta:

```
dart run build_runner build
```

No edites manualmente `models.freezed.dart` ni `models.g.dart`; son archivos generados.

lib/state/auth_state.dart

Gestiona la sesion:

1. Guarda URL y API key.
2. Comprueba la conexion.
3. Valida USR_M.
4. Resuelve el contacto ENT.
5. Comprueba ES_CMR para comerciales.
6. Guarda el usuario actual.
7. Limpia la sesion al desconectar.

El usuario real debe salir de Velneo. No debe volver a introducirse un usuario demo.

lib/core/order_repository.dart

Cache en memoria y filtros locales.

El comercial se filtra comparando:

```
VTA_PED_G.CMR == User.contactId
```

Los filtros de texto buscan en nombre, telefono, CIF y numero de pedido. La busqueda tolera acentos.

lib/core/app_lifecycle_manager.dart

Limpia cache y sesion cuando la aplicacion se destruye o se desconecta.

lib/screens/

Pantallas completas:

- login_screen.dart: formulario de acceso.
- pedidos_list_screen.dart: lista, carga inicial, filtros y busqueda.
- pedido_detail_screen.dart: detalle del pedido.
- pedido_form_screen.dart: alta y edicion.

lib/widgets/

Componentes visuales reutilizables:

- cabecera_view.dart: muestra los datos de cabecera.
- lineas_table.dart: lista las lineas.
- fila_linea.dart: pinta una linea.
- totales_card.dart: base, IVA y total.
- cabecera_form.dart: edita cabecera.
- linea_form_modal.dart: edita una linea.

4. Flujo de login

```
LoginScreen
-> AuthState.conectar()
-> PedidosService.checkConnection()
-> GET_USR_M
-> compara usuario y contraseña
-> lee USR_M.ENT
-> GET ENT_M por id
-> comprueba ENT_M.ES_CMR
-> crea User con contactId
```

Configuracion de campos en lib/core/config.dart:

```
usuarioField = 'name';
passwordField = 'pwd';
usuarioContactoField = 'ent';
usuarioRolField = 'rol';
contactoComercialField = 'es_cmr';
```

Si en Velneo los campos se llaman diferente, cambia solamente estas constantes.

5. Flujo de carga de pedidos

Para un comercial se envia a Velneo:

```
GET VTA_PED_G
  ?filter[cmr]=ID_DEL_CONTACTO
  &page[number]=1
  &page[size]=50
```

Esto es importante: el filtro de permisos se hace en Velneo usando el CMR del pedido, no usando el CLT del cliente.

Despues los pedidos se guardan en OrderRepository y los filtros de estado y texto se ejecutan localmente.

6. Flujo del detalle

```
GET VTA_PED_G/{id}
GET VTA_PED_LIN_G?filter[vta_ped]={id}&page[size]=100
GET ENT_M para completar cliente
GET ENT_M para completar comercial
```

Las tres peticiones auxiliares se ejecutan en paralelo.

El detalle utiliza estos campos principales:

Cabecera VTA_PED_G

```
num_ped  -> numeroPedido
clt       -> clienteId
est       -> estado
tot_ped   -> total
n_doc     -> nDocumento
ser       -> serie
fch       -> fecha
fch_ent   -> previstoPara
fpg       -> formaPago
cmr       -> comercial
dir_env   -> direccionEnvio
email     -> email
obs       -> observaciones
```

Linea VTA_PED_LIN_G

```
art       -> articulo
dsc       -> descripcion
ref_man   -> nReferencia
can_ped   -> cantidad
can_pdt   -> pendiente
pre       -> precio
por_dto   -> dto
imp       -> importe
reg_iva_vta -> tipoIva
est       -> estado
cnc       -> cancelado
fch_ent   -> previstoPara
```

Si imp no llega, se calcula:

```
cantidad * precio - descuento
```

7. Estados

Los estados de pedidos de venta se normalizan en lib/theme/app_theme.dart:

```
P, Pendiente          -> Pendiente  
S, Servido, Recibido  -> Servido  
C, A, Cancelado, Anulado -> Cancelado
```

El repositorio compara los estados por código, no por texto literal.

8. Donde cambiar la interfaz

Para cambiar lo que se muestra en el detalle:

- Cabecera: lib/widgets/cabecera_view.dart.
- Lineas: lib/widgets/fila_linea.dart.
- Totales: lib/widgets/totales_card.dart.

Ejemplo:

```
_Campo('Direccion de envio', pedido.direccionEnvio)
```

Para cambiar el formulario:

- Cabecera: lib/widgets/cabecera_form.dart.
- Linea: lib/widgets/linea_form_modal.dart.
- Guardado: lib/screens/pedido_form_screen.dart.

9. Como añadir un nuevo campo de Velneo

1. Confirma el nombre exacto del campo en Velneo.
2. Añade la propiedad al modelo en models.dart.
3. Añade @JsonKey(name: 'campo_velneo') si el nombre Dart es diferente.
4. Añade el alias en _normalizePedidoJson() o _normalizeLineJson() si hay respuestas antiguas.
5. Ejecuta dart run build_runner build.
6. Usa el campo en el widget correspondiente.
7. Añade un test de contrato en test/widget_test.dart.

10. Como añadir una nueva petición API

En api_service.dart:

```
static Future<List<OpcionMaestra>> miConsulta() async {  
  final json = await _api.get(  
    AppConfig.endpoint('miEndpoint'),  
    params: {'page[size]': 100},  
  );  
  return payloadLista(json).map(_opcionFromRecord).toList();  
}
```

No llares directamente a http desde una pantalla.

11. Problemas frecuentes

El campo aparece vacío

Revisa, en este orden:

1. Nombre real de la columna Velneo.
2. Alias en models.dart.
3. Archivo generado con build_runner.
4. Widget que pinta el campo.
5. Respuesta real de la API.

Aparece un ID en vez de un nombre

El campo es una referencia Velneo. Hay que consultar la tabla relacionada y guardar el nombre en una propiedad como comercialNombre o clienteNombre.

La lista tarda

Revisa:

- page[size].
- Descargas completas de maestros.
- Peticiones secuenciales.
- Filtros que puedan ejecutarse directamente en Velneo.

Error de permisos

La API key necesita permisos sobre la tabla y el metodo HTTP. Un error 403, 405 o un bloque errors de Velneo no se arregla desde Flutter: debe corregirse en la configuracion del API de Velneo.

12. Seguridad importante

La API key no deberia estar publicada dentro de una aplicacion distribuida. Para produccion, utiliza un backend o un mecanismo de tokens temporales.

No guardes contraseñas en logs. No imprimas respuestas completas con datos personales.

13. Rutina recomendada para programar sola

Para cada cambio:

1. Decide que pantalla o capa es responsable.
2. Lee el modelo y el servicio relacionado.
3. Cambia una cosa pequeña.
4. Ejecuta:

```
flutter analyze  
flutter test
```

5. Si cambias modelos:

```
dart run build_runner build  
flutter analyze  
flutter test
```

6. Prueba manualmente:

```
flutter run -d windows
```

7. Comprueba primero login, lista, detalle y guardado.

14. Orden recomendado para seguir aprendiendo Flutter

1. Dart: variables, clases, Future, async/await, listas y mapas.
2. Widgets: StatelessWidget, StatefulWidget, build, setState.
3. Navegacion: rutas y argumentos.
4. Formularios: controllers, validacion y TextFormField.

5. HTTP: peticiones, JSON y errores.

6. Estado: Provider, ChangeNotifier y lectura con watch/read.

7. Modelos: Freezed y json_serializable.

8. Tests: pruebas de modelos, servicios y widgets.

Empieza modificando una pantalla o un widget pequeño. Evita cambiar a la vez API, modelos, estado y UI.

15. Comandos de referencia

```
flutter pub get
flutter analyze
flutter test
dart run build_runner build
flutter run -d windows
flutter clean
```

Si una compilacion queda en un estado extraño:

```
flutter clean
flutter pub get
dart run build_runner build
flutter analyze
flutter test
```

16. Estructura mental del proyecto

```
Velneo API
|
ApiClient (HTTP y JSON)
|
PedidosService (operaciones de negocio API)
|
Models (Pedido, LineaPedido, User, Cliente)
|
AuthState + OrderRepository (sesion y cache)
|
Screens (flujo de pantallas)
|
Widgets (interfaz reutilizable)
```

Este es el mapa que debes seguir cuando quieras entender o modificar una funcionalidad.